

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    printf(stderr, "ERRORE: serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    printf(stderr, "ERRORE: impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Advanced IO

Non-Blocking and Asynchronous IO

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

IO system calls

- ❖ To make IO more efficient, IO cannot be performed synchronously
 - Tasks **must have** the possibility to continue **without** waiting for IO operations to complete
- ❖ Non-synchronous IO can be implemented using different techniques
 - Multi-threading IO
 - Non-blocking IO
 - Asynchronous IO

Solution 1

Solution 2

Solution 3

Solution 1: Multi-Threading IO

❖ In multi-threading IO

- We can use multiple threads to avoid waiting for the results and stop the entire task
- Each thread within the task (in the same or in different processes) may use a synchronous model
 - Each thread is responsible for a sequence of one or more synchronous, **blocking**, IO operations
 - Each thread should have its **own** file or pipe handle
 - As threads run asynchronously with respect to each other if one thread is blocked the other can proceed

Example

```
#include <iostream>
#include <future>
#include <thread>
#include <chrono>
```

The thread function performs IO

```
static std::string getI() {
    std::string answer;
    std::cin >> answer;
    return answer;
}
```

```
int main() {
    std::chrono::seconds timeout(5);
    std::cout << "Read string: " << std::flush;
    std::string answer = "maybe"; //default to maybe
```

Use a task for asynchronous IO

```
    std::future<std::string> future = std::async(getI);
```

Work in parallel

```
    while (future.wait_for(timeout) != std::future_status::ready)
        ... Do Something Else ...
```

Get result

```
    answer = future.get();
    std::cout << "Answer: " << answer << std::endl;
    exit(0);
```

```
}
```

Pros and Cons

- ❖ Multi threading is a “simple” and “standard” feature to implement even for non-blocking IO
- ❖ Unfortunately, with multi-threading
 - The solution to a blocked IO is misleading
 - The IO operation **is still blocking** but the calling thread apparently do not stop
- ❖ To really avoid being blocked forever, we must issue the operation in non-blocking or asynchronous mode
 - Non-blocking and asynchronous IO is usually allowed with low-level primitives, i.e., with the POSIX-UNIX standard

Solution 2: Non-Blocking IO

- ❖ Non-blocking means that if an answer cannot be returned rapidly, the system call returns immediately with an **error** and does nothing

- ❖ Non-blocking IO is used when it is possible to

- Do other unrelated things while waiting for the operation to be done

Polling

- **Check** the availability of the response at the right time

- ❖ In POSIX there are **two ways** to specify non-blocking IO for a given descriptor

- While opening a file descriptor

- After opening a file descriptor

Solution 1
Strategy A

Solution 1
Strategy B

Strategy A

```
int fd;  
...  
fd = open (name, O_WRONLY | ... | O_NONBLOCK );
```

- ❖ In strategy A, we define the non-blocking option **when** we open the file
 - We call **open** to get the file descriptor and specify the **O_NONBLOCK** flag
 - The nonblocking mode will be set **for all** subsequent IO operations

Strategy B

```
#include <fcntl.h>
```

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```

- ❖ In strategy B, we define the non-blocking option **after** the file is opened using function **fcntl**
 - This function has a more general use (see file locking) and it can change the properties of a file already opened

Strategy B

```
#include <fcntl.h>
```

Command
(5 overall)

NULL for now
(used in file locking)

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```

File
descriptor



cmd	Purpose
F_GETFL or F_SETFL	Get/set file status flags
F_DUPFD or F_DUPFD_CLOEXEC	Duplicate an existing descriptor
F_GETFD or F_SETFD	Get/set file descriptor flags
F_GETOWN or F_SETOWN	Get/set asynchronous I/O ownership
F_GETLK, F_SETLK, or F_SETLKW	Get/set record locks

Strategy B

```
#include <fcntl.h>
```

Command
(5 overall)

NULL for now
(used in file locking)

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```

File
descriptor



cmd	Purpose
F_GETFL or F_SETFL	Get/set file status flags

- ❖ To modify the file status flags, we must
 - Fetch the existing flags, modify them, set them back
 - We cannot directly modify them as we can turn off flag bits that were previously set

Example

Set file status flag
(wrapper function)

❖ Fetch the existing flag value and **set** it as desired

```
#include <errno.h>
#include <fcntl.h>
```

File descriptor

File flag, e.g.,
O_NONBLOCK

```
void set_fcntl (int fd, int flags) {
    int val;
    if ((val = fcntl(fd, F_GETFL, 0)) < 0)
        { ... error ... }
```

Get flags

```
    val |= flags;
```

Set the desire flag

```
    if (fcntl(fd, F_SETFL, val) < 0)
        { ... Error ... }
```

```
}
```

Set new flag status
for the descriptor fd

Example

Clear file status flag
(wrapper function)

- ❖ Fetch the existing flag value and **reset (clear)** it as desired

```
#include <errno.h>
#include <fcntl.h>
```

File descriptor

File flag, e.g.,
O_NONBLOCK

```
void clr_fcntl (int fd, int flags) {
    int val;
    if ((val = fcntl(fd, F_GETFL, 0)) < 0)
        { ... error ... }
```

Get flags

```
    val &= ~flags;
```

Clear the desire flag

```
    if (fcntl(fd, F_SETFL, val) < 0)
        { ... Error ... }
```

```
}
```

Set new flag status
for the descriptor fd

Example

- ❖ Use a nonblocking write operation to avoid waiting for the output termination

```
#define N 1000000
```

```
...
```

```
char buf[N];
```

```
int ntowrite, nwrite;
```

```
char *ptr;
```

```
...
```

```
ntowrite = read(STDIN_FILENO, buf, sizeof(buf));
```

```
fprintf(stderr, "Read %d bytes\n", ntowrite);
```

```
...
```

```
set_fnctl(STDOUT_FILENO, O_NONBLOCK);
```

Read a lot of data
from stdin

Read is
blocking

Set stdout to
nonblocking

Example

Note: With non-blocking IO we must perform some form of **polling** !!!

Try to write a lot of data to stdout

```
ptr = buf;
while (ntowrite > 0) {
    nwrite = write (STDOUT_FILENO, ptr, ntowrite);

    fprintf (stderr, "Write %d bytes\n", nwrite);
    if (nwrite > 0) {
        ptr += nwrite;
        ntowrite -= nwrite;
    }
}
```

Display output status

Check output status

... Do Something Else ...

Do some other job

Cycle

```
clr_fcntl(STDOUT_FILENO, O_NONBLOCK);
```

Clear stdout from nonblocking

Example

If the standard output is a regular file, we expect the fully perform the writing

```
➤ ./a.out < inFile.txt > outFile.txt  
read 1000000 bytes  
nwrite = 1000000  
➤ ls -l outFile.txt  
-rw-rw-r-- 1 sar 1000000 Apr 1 13:03 outFile.txt
```

If the standard output is a terminal, we expect the write to return a partial count sometimes and an error at other times

```
➤ ./a.out < inFile.txt 2> stderr.txt  
➤ cat stderr.txt  
read 1000000 bytes  
nwrite = 999  
nwrite = -1  
nwrite = 1001  
nwrite = -1  
nwrite = 1002  
...
```

Pros and Cons

- ❖ Non-blocking IO is straightforward to implement
- ❖ Unfortunately, it implies some polling to know how the operation proceeds
 - In other words, there is no way to be asynchronously woken up when the process ends
 - Polling, per se,
 - It is expensive
 - It is hard to establish how often it has to be performed

Solution 3: Asynchronous IO

- ❖ Similarly to non-blocking IO, in asynchronous IO the program does other unrelated things while it waits for the operation to be done
- ❖ However, instead of checking on the result of the IO operation, the program delegates the work of checking to something else
 - The program is **notified** and possibly **interrupted** when the response is available
 - There is **no polling** involved
- ❖ Unfortunately, several **implementation details** make the process **very complex**

Solution 3: Asynchronous IO

- ❖ The additional complexity is related to the POSIX asynchronous IO interface
 - The interface involves a lot of **extra setup** and processing rules compared to their conventional counterparts
 - We have to worry about sources of errors for every asynchronous operation
 - Recovering from errors can be difficult
 - For example, if we submit multiple asynchronous writes and one fails, how should we proceed?
 - If the writes are related, we might have to undo the ones that succeeded

Solution 3: Asynchronous IO

- ❖ The POSIX interface uses
 - An **AIO control block** to describe the IO operations
 - Structure `aiocb`
 - A **signal event control block** to specify how to indicate the termination of the IO operation
 - Structure `sigevent`
 - A **set of dedicated system calls** to perform the IO operation, to check its status, wait for its conclusion, etc.
 - System calls `aio_read`, `aio_write`, `aio_fsync`, `aio_error`, `aio_return`, `aio_suspend`, `aio_cancel`

The aiocb data structure

❖ The AIO control block

- Is implementation dependent
- It contains at least the fields shown in the following structure
 - Some implementations might include additional fields
- To use it, we must
 - Insert the `<aio.h>` library in the file

```
#include <aio.h>
```

- Compile the C program with the realtime library (librt.a), i.e., `-lrt`, at compilation time

```
gcc -Wall -o <...> <...>.c -lrt
```


The aiocb data structure

```
#include <aio.h>
```

```
struct aiocb {  
    int aio_fildes;  
    off_t aio_offset;  
    volatile void *aio_buf;  
    size_t aio_nbytes;  
    int aio_reqprio;  
    struct sigevent aio_sigevent;  
    int aio_lio_opcode;  
};
```

Defines the file descriptors, the file offsets, the buffer for IO, the number of bytes to transfer, etc.

Control how the application is notified about the completion of IO events

```
struct sigevent {  
    int sigev_notify;  
    int sigev_signo;  
    union sigval sigev_value;  
    void (*sigev_notify_function)(union sigval);  
    pthread_attr_t *sigev_notify_attributes;  
};
```

The aiocb data structure

- ❖ The meaning of the fields within the AIO control block is the following

Field	Meaning
aio_fildes	The file descriptor open for the file to be read or written.
aio_offset	Offset from where to start the read or write operation.
aio_buf	Pointer for the IO data buffer. Data is copied from the buffer during a read and into the buffer during a write.
aio_nbytes	Specifies the number of bytes to read or write.
aio_reqprio	A hint to suggest an ordering (i.e., a priority) for the asynchronous IO requests.
aio_sigevent	Controls how the application is notified about the completion of the IO event. It is described by a sigevent structure
aio_lio_opcode	This field is used only with list-based operation IO.

The aioctx data structure

- ❖ The field `sigev_notify`, inside the `sigevent` structure, controls the type of notification

Operation	Meaning
SIGEV_NONE	The process is not notified when the asynchronous IO request completes.
SIGEV_SIGNAL	The signal specified by the <code>sigev_signo</code> field is generated when the asynchronous IO request completes.
SIGEV_THREAD	The function specified by the <code>sigev_notify_function</code> field is called when the asynchronous IO request completes. The function is executed in a separate thread in a detached state.

Asynchronous IO system calls

Operation	Meaning
<code>int aio_read(struct aiocb *aiocb);</code> <code>int aio_write(struct aiocb *aiocb);</code>	Perform asynchronous IO. The AIO control block must be initialized before the operation. When these functions return success, the asynchronous IO request has been queued for processing by the operating system.
<code>int aio_fsync (int op, struct aiocb *aiocb);</code>	Force all pending asynchronous writes to persistent storage without waiting.
<code>int aio_error(const struct aiocb *);</code>	Determine the completion status of any asynch operation.
<code>int aio_return(const struct aiocb *);</code>	Return the operation result.
<code>int aio_suspend(const struct aiocb *const list[], int nent, const struct timespec *timeout);</code>	Block the task until an asynch IO operation completes.
<code>int aio_cancel (int fd, struct aiocb *aiocb);</code>	Attempt to cancel pending async IO operations that we do not want to execute anymore.

Example

- ❖ Write a program that
 - Asynchronously reads a string from standard input
 - Waits (pause) until the string has been read and it is woken up by a signal
 - Iterates through the two previous steps until the string is equal to "end"

Example

Desired behavior

Execution example

```
quer@quer-VB:~/tmp1$ ./pgrm
Waiting for input ... 123
Received SIGUSR1; Received input: 123
Waiting for input ... 45
Received SIGUSR1; Received input: 45
Waiting for input ... AA
Received SIGUSR1; Received input: AA
Waiting for input ... end
Received SIGUSR1; Received input: end
```

Main cycle

```
while (1) {
    if (aio_read(&aio_cb) == -1) {
        fprintf ("Erron in aio_read.\n");
        exit(EXIT_FAILURE);
    }
    printf("Waiting for input ... ");
    pause();
}

return 0;
}
```

Async read

Suspend the process until a signal is received (i.e., an input is read)

Initialization

Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <aio.h>
#include <errno.h>
#include <string.h>

#define BUFFER_SIZE 128

struct aiocb aio_cb;
```


The signal handler

Example

```
void aio_completion_handler(  
    int sig, siginfo_t *si, void *context) {  
    if (sig == SIGUSR1) {  
        printf("Received SIGUSR1; ");  
        if (aio_error(&aio_cb) == 0) {  
            ssize_t bytes_read = aio_return(&aio_cb);  
            if (bytes_read > 0) {  
                char *str = (char *) aio_cb.aio_buf;  
                str[bytes_read-1] = '\\0';  
                printf("Received input: %s\\n", str);  
                if (strcmp(str, "end")==0)  
                    exit (1)  
            } else {  
                printf("No input received\\n");  
            }  
        } else {  
            printf ("aio_error");  
        }  
    }  
}
```

Get the
number of
bytes read

Check if the asynchronous
read completed successfully

Null-terminate
the buffer

End the program

The main program

Example

```
int main() {
    struct sigaction sa;
    char buffer[BUFFER_SIZE];

    setbuf (stdout, 0);

    memset(&sa, 0, sizeof(sa));
    sa.sa_flags = SA_SIGINFO;
    sa.sa_sigaction = aio_completion_handler;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
        printf ("Error on sigaction.\n");
        exit (EXIT_FAILURE);
    }
}
```

Set up the
signal handler
for SIGUSR1

System call sigaction is
an extended (updated)
version of signal

Example

```
memset(&aio_cb, 0, sizeof(struct aiocb));
aio_cb.aio_fildes = STDIN_FILENO;
aio_cb.aio_buf = buffer;
aio_cb.aio_nbytes = BUFFER_SIZE - 1;
aio_cb.aio_sigevent.sigev_notify = SIGEV_SIGNAL;
aio_cb.aio_sigevent.sigev_signo = SIGUSR1;

while (1) {
    if (aio_read(&aio_cb) == -1) {
        printf ("Erron in aio_read.\n");
        exit(EXIT_FAILURE);
    }
    printf ("Waiting for input ... ");
    pause();
}

return 0;
}
```

Initialize the aiocb structure

Async read

Suspend the process until a signal is received (i.e., an input is read)